

Mewtual - Documentation

Joan Camilo Betancourt González

David Eduardo Calderón Parra

Sergio Andrés Hernández Salinas

Daniel Oñate Hernández

Raul Felipe Rodríguez Hernández



Ingeniería de Software II

Facultad de Ingeniería

Universidad Nacional de Colombia

Liliana Marcela Olarte Mesa

29 de julio de 2026

1. Introduction

1.1 System Description

Mewtual is a web application designed for cat owners who wish to find compatible partners for their pets with the goal of facilitating responsible and informed reproduction. The platform provides tools to create detailed cat profiles, manage health records, and discover compatible cats through a swipe-based matching system.

Once a match is established between two cats, the platform enables communication between their owners via an integrated chat, allowing them to coordinate meetings, share additional information, and plan the mating process responsibly.

1.2 Motivation and Need

- Allows owners to verify health and vaccination status before any interaction.
- Ensures only cats meeting responsible mating conditions (vaccinated, unsterilized, with a medical certificate) can participate in the matching system.
- Simplifies the search for compatible partners using filters such as breed, age, and health status.
- Facilitates post-mating follow-up including adoption publication.

1.3 Actors

Actor	Description
Cat Owner (User)	Primary actor. Registers cats, uses the matching system, chats with other owners, and manages the reproduction lifecycle.
Administrator (Admin)	Manages the platform: suspends accounts, verifies profiles, reviews reports, and sends mass notifications.

2. System Architecture

2.1 Architectural Style

Mewtual follows a Modular Monolith architecture using Django as the web framework. The application is organized into independent Django apps, each responsible for a specific domain. Each app applies the Model-View-Controller (MVC) pattern internally: models define the database schema, views handle business logic and HTTP routing, and templates render the HTML frontend. No app accesses another app's internals directly – cross-app communication goes through model relationships and the Django ORM.

2.2 Module Structure

Module (App)	Responsibility
users	Authentication, registration, login/logout, user profiles, account deletion, password reset.
cats	Cat profile management (create, edit, delete), photo gallery, vaccination records, medical records, responsible mating validation.
matching	Swipe logic (like/dislike), match generation (pending → accepted), candidate filtering.
chat	Messaging between matched users, image sharing, adoption-initiated chats. Auto-created via Observer signal on Match acceptance.
appointments	Scheduling and confirming meetings between cat owners (partially implemented).
adoption	Publishing cats for adoption or gifting, tracking adoption status (AVAILABLE / RESERVED / ADOPTED).
reports	Behavior reporting between users, admin review workflow.
config	Orchestrates all apps: INSTALLED_APPS, database connection, media configuration, URL routing entry point.

2.3 Technology Stack

Layer	Technology	Purpose
Backend	Python 3.12 + Django 6.0	Web framework, ORM, authentication, URL routing, template engine.
REST API	Django REST Framework 3.17	ModelViewSet + DefaultRouter for all API endpoints. Session-based authentication (same cookie as web login).
Database	PostgreSQL 16	Relational database. Referential integrity via FK constraints and CASCADE deletes.
Containerization	Docker + Docker Compose	PostgreSQL runs in a container – one command setup for all team members.
Frontend	Django Templates + Tailwind CSS (CDN)	Server-side rendered HTML. No separate React/Vue frontend.
Design	Figma	UI mockups designed in Figma, then translated to HTML/Tailwind.
Image handling	Pillow 12.3	Processing cat photos and profile pictures (ImageField).
Environment	python-dotenv 1.2	Loading .env credentials outside source code.
Linting	Flake8	PEP 8 style checking.
CI/CD	GitHub Actions	Automated tests + migrations on every push/PR to main.

Layer	Technology	Purpose
Version control	Git + GitHub	Source code management. Repository: github.com/rrodriguezhe/mewtual

2.4 Project Structure

The repository follows this folder layout:

```

mewtual/
Final-Project/
  backend/      ← manage.py lives here
  config/       ← settings.py, urls.py (entry point)
  users/        ← auth, profiles, account deletion
  cats/         ← cat profiles, gallery, vaccines, medical records
  matching/     ← swipe, like/dislike, match generation
  chat/         ← messaging, signals.py (Observer)
  appointments/ ← meeting scheduling (partial)
  adoption/     ← adoption posts, IsGatoOwnerOrStaff (Strategy)
  reports/      ← reports, get_permissions (Strategy)
  media/        ← uploaded files
  .github/workflows/ci.yml ← CI/CD pipeline
  .env          ← credentials (not committed)

```

2.5 Data Flow Overview

- The browser sends an HTTP request to the Django development server.
- `config/urls.py` dispatches it to the corresponding app's `urls.py`.
- The view function handles business logic, queries the database via the ORM, and prepares context.
- For template views: Django renders an HTML page styled with Tailwind CSS.
- For API views: Django REST Framework serializes the response to JSON via `ModelViewSet`.
- Media files (cat photos, vaccination documents, medical certificates) are served from `/media/`.
- On every Match save with `estado="ACEPTADO"`, the Observer signal in `chat/signals.py` auto-creates the Chat (see Section 4).

3. Database Schema

3.1 users app

Model	Key Fields	Description
User (Django built-in)	id, username, email, password, is_active	Core authentication model. Account deletion sets is_active=False instead of deleting the row.
Profile	user (FK→User), foto_perfil, bio, first_name, last_name	Extended user information and profile picture.
Block	blocker (FK→User), blocked (FK→User), created_at	Records of users who have blocked other users.

3.2 cats app

Model	Key Fields	Description
Cat	owner (FK→User), nombre, raza, sexo, fecha_nacimiento, peso, color, foto, gustos_preferencias, esterilizado, modo_cruce_responsable, fecha_registro	Core cat profile. foto_principal property returns first gallery photo or falls back to foto field.
CatPhoto	gato (FK→Cat), imagen, orden	Multiple ordered photos per cat (gallery feature).
Vaccine	gato (FK→Cat), nombre, fecha_aplicacion, fecha_vencimiento, soporte	Vaccination records. Expiry drives responsible mating eligibility.
MedicalRecord	gato (FK→Cat), diagnostico_procedimiento, notas, documento, fecha_registro	Veterinary certificates. Document attachment required for responsible mating mode.
Favorite	usuario (FK→User), gato (FK→Cat), fecha_guardado	Model defined; UI not yet implemented.

3.3 matching app

Model	Key Fields	Description
Like	from_cat (FK→Cat), to_cat (FK→Cat), created_at	Stores swipe decisions.
Dislike	from_cat (FK→Cat), to_cat (FK→Cat), created_at	Stores rejections (excluded from future swipe stacks).
Match	cat_a (FK→Cat), cat_b (FK→Cat), estado (PENDIENTE/ACEPTADO/RECHAZADO), created_at	Created when one cat likes another. Transitions to ACEPTADO when the other cat likes back, triggering Chat creation via Observer signal.

3.4 chat app

Model	Key Fields	Description
Chat	match (FK→Match, nullable), created_at	Conversation thread. Auto-created by Observer signal when Match reaches ACEPTADO. Can also be started directly from an adoption post.
Message	chat (FK→Chat), sender (FK→User), content, image, sent_at	Individual messages. Supports text and image attachments. Polled every 3 seconds by the JS chat client.

3.5 adoption app

Model	Key Fields	Description
AdoptionPost	gato (FK→Cat), descripcion, estado (AVAILABLE/RESERVED/ADOPTED), publicado_en	Adoption listing. IsGatoOwnerOrStaff (Strategy pattern) restricts write access to owner or staff only.

3.6 reports app

Model	Key Fields	Description
Report	usuario_reportante (FK→User), usuario_reportado (FK→User), motivo, estado (PENDING/REVIEWED/CLOSED), fecha_reporte	Behavior report. get_permissions() Strategy controls who can update estado (staff only).

3.7 Key Relationships

- A User can own multiple Cats (one-to-many, CASCADE delete).
- A Cat can have multiple CatPhotos, Vaccines, and MedicalRecords (one-to-many, CASCADE delete).
- A Match is created when one Cat likes another (PENDIENTE); transitions to ACEPTADO when the like is returned, triggering Chat creation.
- A Chat is linked to a Match (matching flow) or created directly from an AdoptionPost.
- A User can block other Users via the Block model.

4. Design Patterns

Two design patterns were identified and implemented in the codebase. Both arise from real architectural constraints rather than being retrofitted.

4.1 Pattern 1 – Strategy (Behavioral)

Problem

Both the adoption and reports apps need different authorization rules depending on the action being performed and the role of the requester. For example, in adoption, reads must be open to any authenticated user but writes are restricted to the cat's owner or staff; in reports, only staff can transition a report's estado field, while any user can file one. Hard-coding these checks as if/elif chains inside each view method would tie authorization logic to CRUD logic and make each rule harder to test or swap independently.

Solution

Each authorization rule is encapsulated as an interchangeable object behind DRF's common BasePermission interface, selected at runtime:

IsGatoOwnerOrStaff – adoption/views.py

Implements has_object_permission: reads are open to any authenticated user; writes are restricted to the cat's owner or staff. The "algorithm" (owner-or-staff-can-write) is encapsulated in one class and reused across the adoption ViewSet without branching in each method.

```
class IsGatoOwnerOrStaff(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user and request.user.is_authenticated

    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:
            return True
        return obj.gato.owner == request.user or request.user.is_staff
```

get_permissions() – reports/views.py

Selects a different permission strategy (IsAuthenticated vs. IsAdminUser) depending on self.action, rather than branching inline in each method. This is DRF's own Strategy hook, extended for per-action variation.

```
def get_permissions(self):
    if self.action in ["update", "partial_update", "destroy"]:
        return [IsAdminUser()]
    return [IsAuthenticated()]
```

get_fields() – reports/serializers.py

Swaps field-level read-only behavior for the estado field based on the requester's role. Regular users cannot set estado on creation or update; staff users can.

Where it is applied

- Final-Project/backend/adoption/views.py → IsGatoOwnerOrStaff
- Final-Project/backend/reports/views.py → ReportViewSet.get_permissions()
- Final-Project/backend/reports/serializers.py → ReportSerializer.get_fields()

Tests

- Final-Project/backend/adoption/tests.py – exercises owner vs. non-owner vs. staff per action.
- Final-Project/backend/reports/tests.py – exercises each permission strategy outcome.

4.2 Pattern 2 – Observer (Behavioral)

Problem

When a Match transitions to estado = "ACEPTADO" (mutual like), HU-09 and HU-12 require that a Chat be created automatically for both users without either of them taking an extra action. The matching app owns the swipe/accept flow and must not import from chat – the dependency is strictly one-directional (matching → chat). Calling Chat creation directly from inside the Match model or matching views would invert that dependency.

Solution

The Match model is the subject – it saves itself and knows nothing about chats. The chat app subscribes as an observer via Django's built-in signal dispatcher:

```
# chat/signals.py
from django.db.models.signals import post_save
from django.dispatch import receiver
from matching.models import Match
from .models import Chat

@receiver(post_save, sender=Match)
def crear_chat_al_aceptar(sender, instance, **kwargs):
    if instance.estado == "ACEPTADO":
        Chat.objects.get_or_create(match=instance)
```

The signal is registered when the ChatConfig.ready() lifecycle hook runs, wired in chat/apps.py:

```
# chat/apps.py
class ChatConfig(AppConfig):
    name = "chat"

    def ready(self):
        import chat.signals # noqa: F401
```

Any Match save that flips estado to "ACEPTADO" triggers chat creation automatically. get_or_create keeps it idempotent if the match is saved again while already accepted. The dependency stays one-directional: chat imports matching.models.Match, never the reverse.

Where it is applied

- Final-Project/backend/chat/signals.py → crear_chat_al_aceptar
- Final-Project/backend/chat/apps.py → ChatConfig.ready()

Tests

- Final-Project/backend/matching/tests.py →
test_like_with_incoming_pending_match_becomes_mutual: mutual like flips estado to ACEPTADO and asserts the Chat now exists.
- Final-Project/backend/matching/tests.py →
test_like_with_no_incoming_match_creates_pending_match: still PENDIENTE, no chat yet.
- Final-Project/backend/matching/tests.py →
test_rechazar_with_incoming_pending_match_marks_it_rechazado: RECHAZADO, no chat created.

4.3 OOP Principles Applied

- Encapsulation: Each Django app encapsulates its own models, views, forms, serializers, and URLs. No app accesses another app's internals directly.
- Abstraction: ModelForms (CatForm, VaccineForm, MedicalRecordForm) abstract validation and database mapping from the views.
- Inheritance: All form classes inherit from forms.ModelForm. IsGatoOwnerOrStaff inherits from permissions.BasePermission, reusing DRF's permission interface.
- Polymorphism: DRF's ModelViewSet provides a consistent interface (list, create, retrieve, update, destroy) across all resources. get_permissions() and get_fields() override base behavior polymorphically per action and per role.

5. Implemented User Stories

The following user stories have been implemented and are demonstrable in the current version of the application.

HU-01 – User Registration

As: a cat owner

I need: to register on the platform with my email and a password

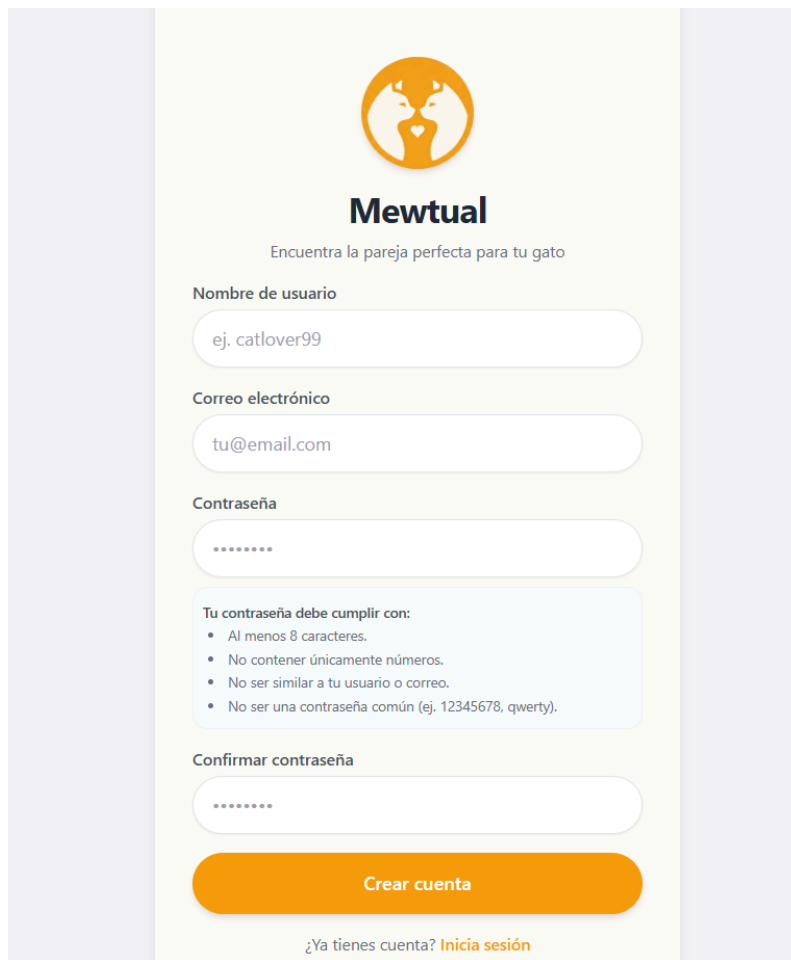
So that: I can access all Mewtual features

Status:  **Implemented**

Module: `users/views.py` → `register_view`

Evidence:

The `register_view` validates that all fields are present, passwords match, username and email are not already taken, and the password meets Django's minimum security requirements. Upon successful registration, the user is redirected to the login screen with a confirmation message.



The image shows a web registration form for 'Mewtual'. At the top is an orange circular logo with a cat face and a heart. Below the logo is the text 'Mewtual' and the tagline 'Encuentra la pareja perfecta para tu gato'. The form contains four input fields: 'Nombre de usuario' (with the example 'ej. catlover99'), 'Correo electrónico' (with the example 'tu@email.com'), 'Contraseña' (masked with dots), and 'Confirmar contraseña' (also masked with dots). Below the password fields is a light blue box with the heading 'Tu contraseña debe cumplir con:' followed by four bullet points: 'Al menos 8 caracteres.', 'No contener únicamente números.', 'No ser similar a tu usuario o correo.', and 'No ser una contraseña común (ej. 12345678, qwerty)'. At the bottom of the form is an orange button labeled 'Crear cuenta'. Below the button is a link that says '¿Ya tienes cuenta? Inicia sesión'.

Figure 1. Register account screen.

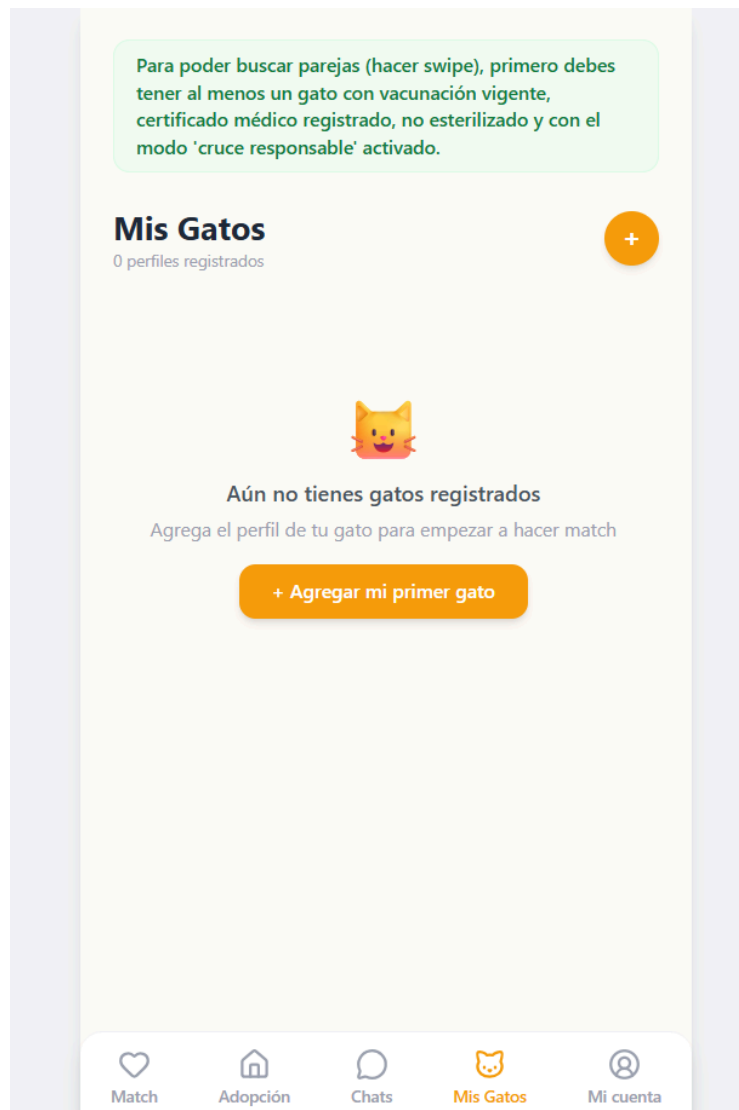


Figure 2. Home screen with the registered account.

HU-02 – Login

As: a registered cat owner

I need: to log in with my email and password

So that: I can access my account and cat profiles

Status:  **Implemented**

Module: `users/views.py` → `login_view`

Evidence:

The `login_view` accepts either email or username. If an email is provided, it first resolves the corresponding username before calling `authenticate()`. On success, the session is established and the user is redirected to the home screen.



Mewtual

Encuentra la pareja perfecta para tu gato

Nombre de usuario o correo electrónico

Contraseña

[¿Olvidaste tu contraseña?](#)

Iniciar sesión

[¿No tienes cuenta? Crear cuenta](#)

Figure 3. Log in screen.

The image shows a login screen for an application called 'Mewtual'. At the top, there is an orange circular logo featuring two cat faces facing each other with a heart shape between them. Below the logo, the word 'Mewtual' is written in a bold, black, sans-serif font. Underneath the name, the tagline 'Encuentra la pareja perfecta para tu gato' is displayed in a smaller, grey font. The main form area contains two input fields: the first is labeled 'Nombre de usuario o correo electrónico' and contains the text 'usuariodemo'; the second is labeled 'Contraseña' and contains a series of dots representing a masked password. To the right of the password field, there is a link that says '¿Olvidaste tu contraseña?'. Below these fields is a large, rounded orange button with the text 'Iniciar sesión'. At the bottom of the form, there is a link that says '¿No tienes cuenta? Crear cuenta'.

Figure 4. Log in screen completed.

HU-03 – Edit User Profile

As: a cat owner

I need: to edit my profile information (name, photo)

So that: my data is up to date

Status:  **Implemented**

Module: `users/views.py` → `mi_cuenta`

Evidence:

The `mi_cuenta` view renders a form pre-populated with the user's first name, last name and profile photo. Username and email are displayed as read-only. A "Delete account" panel requires password confirmation before deactivating.

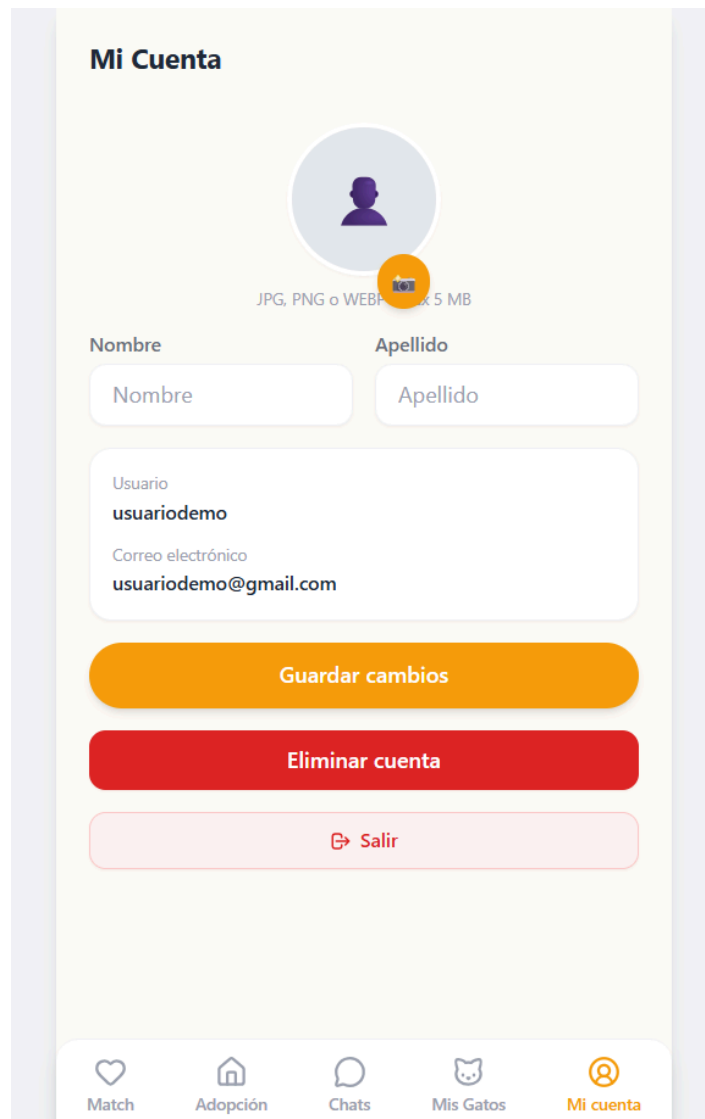


Figure 5. Mi cuenta screen letting edit user's profile.

HU-04 – Cat Profile Creation

As: a cat owner

I need: to create a profile for my cat with basic information (name, breed, age, photos)

So that: other users can learn about my cat and evaluate compatibility

Status:  **Implemented**

Module: `cats/views.py` → `crear_perfil` · `cats/forms.py` → `CatForm`

Evidence:

The CatForm validates required fields (name, breed, birth date, sex), enforces maximum image size (5 MB), and only accepts JPG, PNG, or WEBP formats. The cat is automatically linked to the authenticated user. A vaccination record and medical certificate can optionally be added during creation.

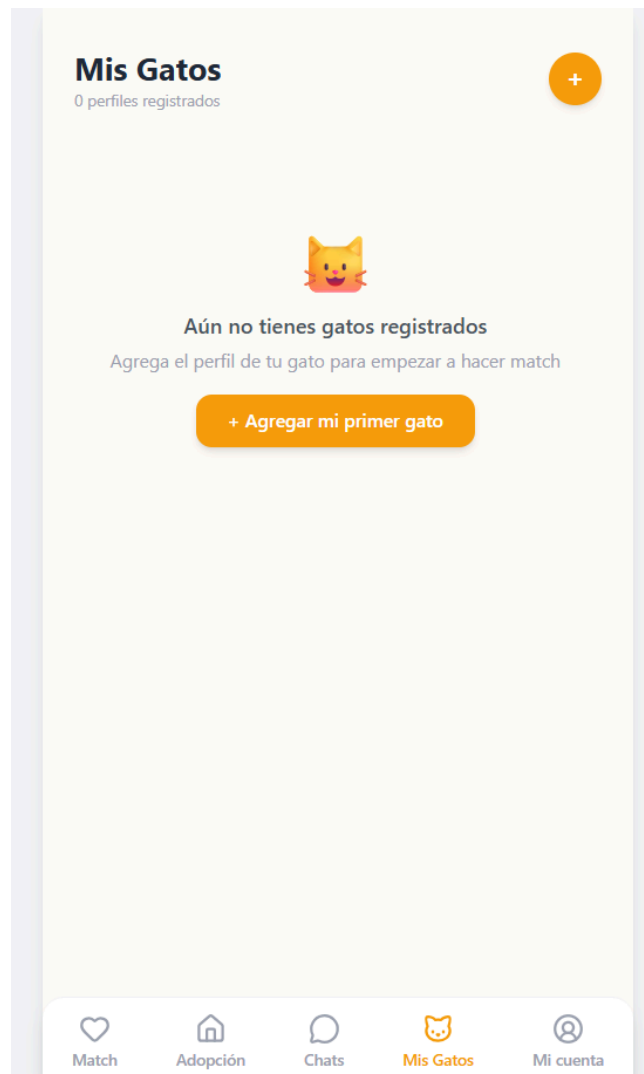
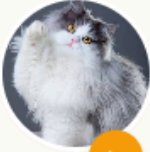


Figure 6. Mis Gatos screen without cats profiles created.

← **Nuevo Gato**



JPG, PNG o WEBP. Máx. 5 MB

Nombre *

Liam

Nacimiento * Peso (kg)

30/02/2020  4

Género *

----- **Macho** Hembra

Raza *

Persa

Color de pelaje

Blanco y gris

Descripción

Juguetón y cariñoso

Figure 7. Nuevo Gato screen that contains the form to create a cat profile P1.

☒ Esterilizado

☒ Modo cruce responsable

Vacuna inicial (opcional)
 Nombre de la vacuna
 Rabia

Fecha aplicación: 30/06/2026
 Fecha vencimiento: 30/06/2027

Certificado médico (opcional) Cruce responsable
 Requerido para habilitar el modo cruce responsable. Debe ser emitido por un veterinario.

Diagnóstico o procedimiento
 Revisión general

Notas adicionales
 Observaciones adicionales del veterinario (opcional)...

Adjuntar certificado (PDF o imagen)
 Seleccionar archivo Certificado ...ico Liam.pdf

Crear perfil

Figure 8. Nuevo Gato screen that contains the form to create a cat profile P2.

HU-Extra-1 — See My Cat Profiles

As: a cat owner

I need: to see the list of cat profiles I have created

So that: I can check their status at a glance

Status: ✔ **Implemented**

Module: [cats/views.py](#) → [mis_gatos](#)

Evidence:

The `mis_gatos` view queries `Cat.objects.filter(owner=request.user)`. For each cat it calculates age (`calcular_edad()`), vaccination status (`vacunas_vigentes()`), and responsible mating eligibility (`puede_cruce_responsable()`). Each card shows name, breed, age, sex, vaccination badge, and mating mode status.

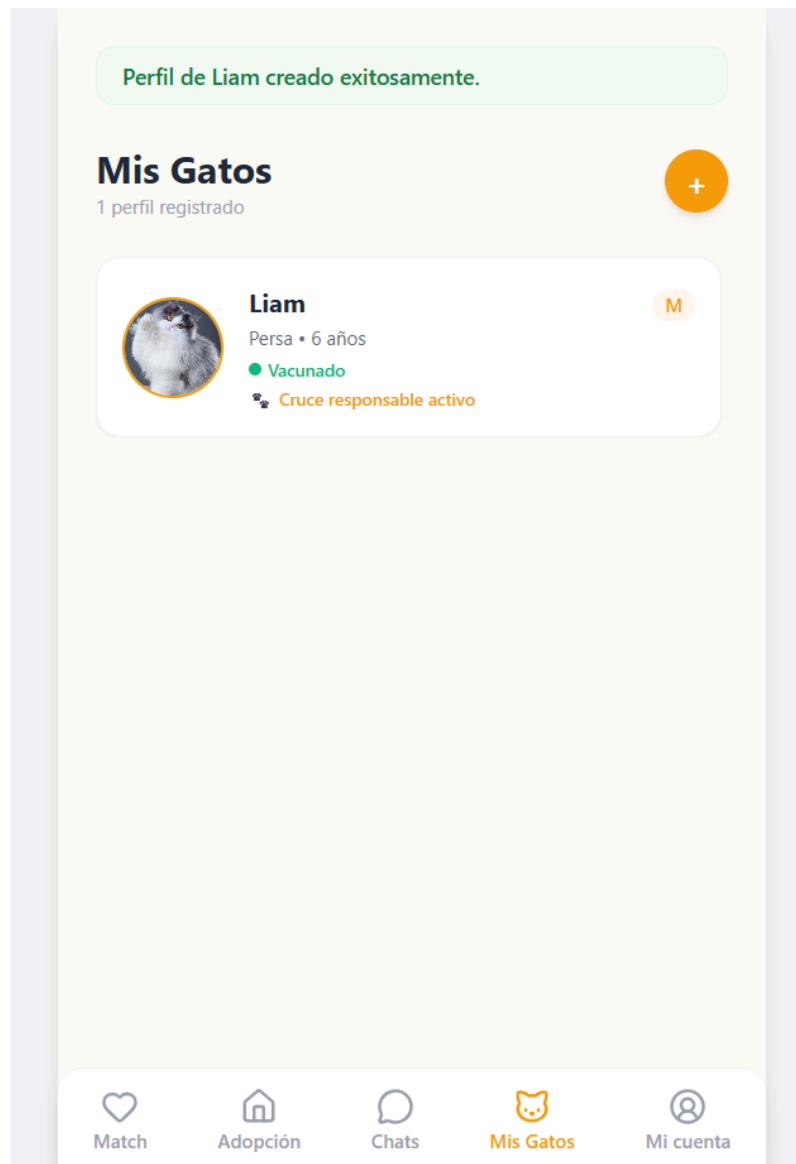


Figure 9. Mis Gatos screen that shows the profiles I have created.

HU-Extra-2 – Edit My Cat Profile

As: a cat owner

I need: to edit a cat profile I have already created

So that: I can keep my cat's data up to date

Status: ☒ **Implemented**

Module: `cats/views.py` → `editar_perfil` · `cats/forms.py` → `CatForm`, `VaccineForm`, `MedicalRecordForm`

Evidence:

`get_object_or_404(Cat, pk=cat_id, owner=request.user)` enforces ownership. `VaccineForm` and `MedicalRecordForm` are optional — silently skipped if left blank. A real-time status panel shows which of the three responsible mating conditions are met.

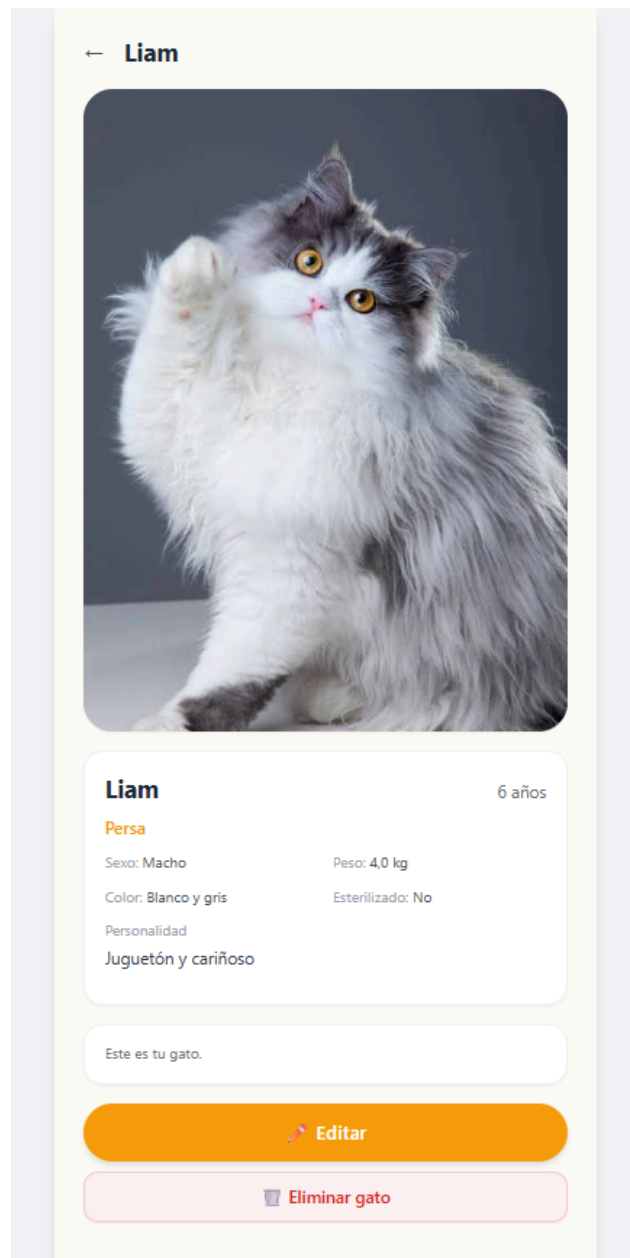


Figure 10. Information screen of a cat profile.

←

Editar Perfil

Fotos del gato



Principal



Agregar

JPG, PNG o WEBP · Máx 5 MB cada una · Máx 6 fotos en total

La primera foto es la que verán los demás. Arrástrala para cambiar el orden.

Nombre *

Liam

Nacimiento *

28/02/2020



Peso (kg)

4,0

Edad actual: 6 años

Género *

Macho

Hembra

Raza *

Persa

Color de pelaje

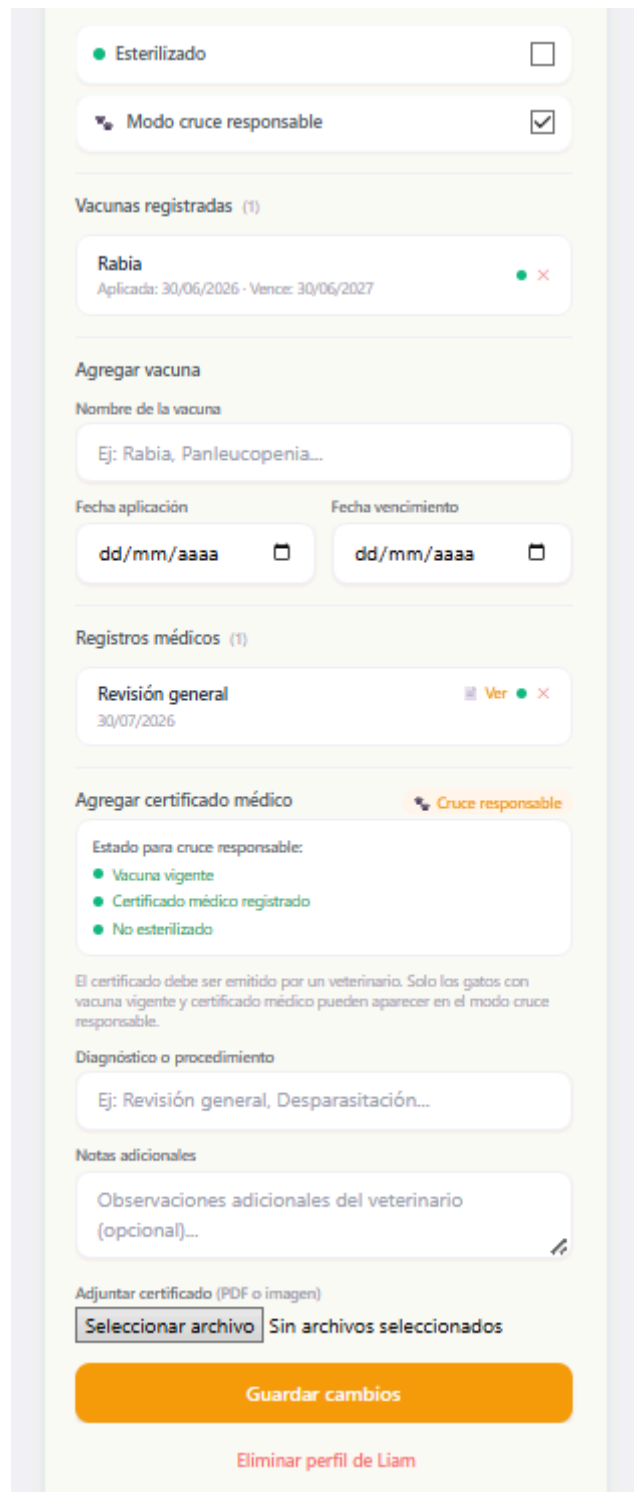
Blanco y gris

Descripción

Juguetón y cariñoso

Figure 11. Editar Perfil screen that contains the form to edit a cat profile P1.

19



☒ Esterilizado ☐

☒ Modo cruce responsable ☐

Vacunas registradas (1)

Rabia ● ✕
 Aplicada: 30/06/2026 - Vence: 30/06/2027

Agregar vacuna

Nombre de la vacuna

Ej: Rabia, Panleucopenia...

Fecha aplicación Fecha vencimiento

dd/mm/aaaa dd/mm/aaaa

Registros médicos (1)

Revisión general 📄 ● ✕
 30/07/2026

Agregar certificado médico Cruce responsable

Estado para cruce responsable:

- ☒ Vacuna vigente
- ☒ Certificado médico registrado
- ☒ No esterilizado

El certificado debe ser emitido por un veterinario. Solo los gatos con vacuna vigente y certificado médico pueden aparecer en el modo cruce responsable.

Diagnóstico o procedimiento

Ej: Revisión general, Desparasitación...

Notas adicionales

Observaciones adicionales del veterinario (opcional)...

Adjuntar certificado (PDF o imagen)

Sin archivos seleccionados

Guardar cambios

[Eliminar perfil de Liam](#)

Figure 12. Editar Perfil screen that contains the form to edit a cat profile P2.

HU-Extra-3 – Delete My Cat Profile

As: a cat owner

I need: to permanently delete a cat profile I have created

So that: I can remove cats that are no longer available

Status: ✔ **Implemented**

Module: `cats/views.py` → `eliminar_perfil` · `cats/templates/cats/confirmar_eliminar.html`

Evidence:

Deletion only executes on POST, preventing accidental removal via direct URL. The confirmation screen warns that vaccines and medical records will also be deleted (CASCADE). After deletion the user is redirected to `mis_gatos` with a confirmation message.

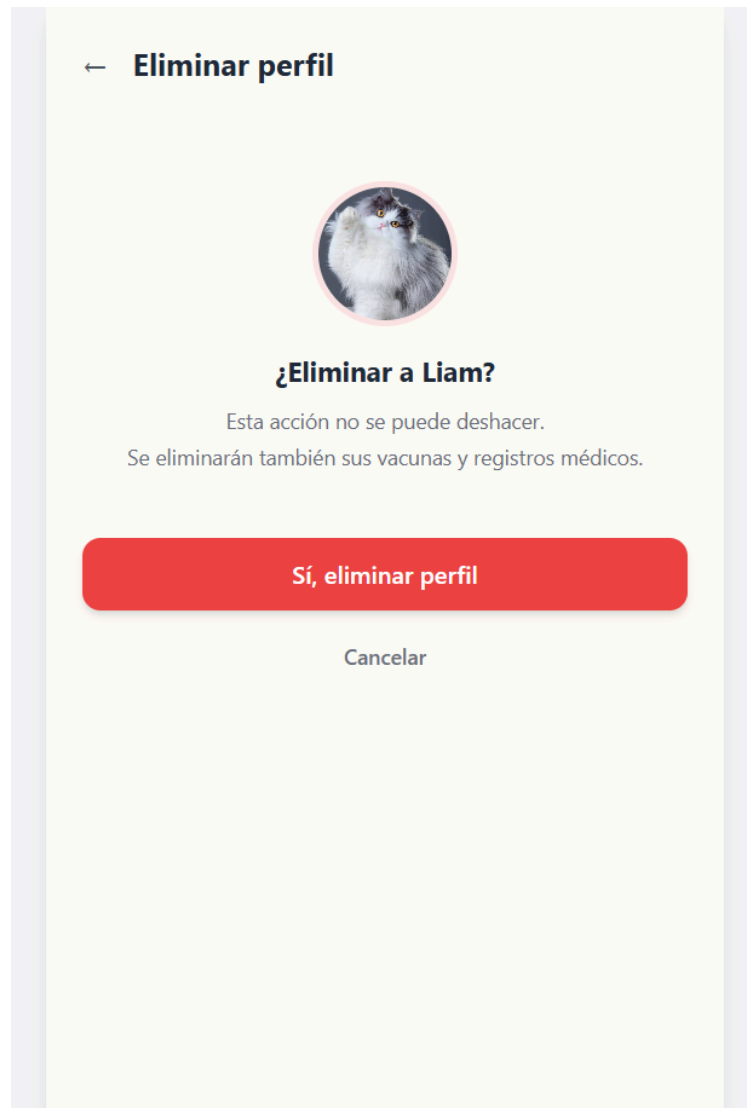


Figure 13. Eliminar Perfil screen that contains the button to delete a cat profile.

HU-05 – Vaccination Record

As: a cat owner

I need: to register my cat's vaccines on its profile

So that: other users can verify vaccination status

Status:  **Implemented**

Module: `cats/views.py` → `editar_perfil` · `cats/forms.py` → `VaccineForm`

Evidence:

VaccineForm is fully optional – if the name field is left blank the form is silently skipped. If a name is provided, application and expiry dates become required, with cross-field validation ensuring expiry is after application date. Existing vaccines are listed with a link to the supporting document and a delete option.



Vacunas registradas (1)

Rabia ● ✕
 Aplicada: 30/06/2026 · Vence: 30/06/2027

Agregar vacuna

Nombre de la vacuna

Triple felina

Fecha aplicación: 30/06/2026 📅

Fecha vencimiento: 30/06/2027 📅

Figure 14. Editar Perfil screen with a new vaccine added.

HU-06 – Responsible Mating Mode

As: a cat owner

I need: the system to validate minimum health conditions before appearing in matching

So that: a safe mating is guaranteed for both cats

Status: ✅ **Implemented**

Module: `cats/views.py` → `puede_cruce_responsable` · `cats/forms.py` → `CatForm.clean()`

Evidence:

`puede_cruce_responsable()` checks: (1) at least one non-expired vaccine, (2) at least one MedicalRecord with attached document, (3) cat is not sterilized. All three must be true for the cat to appear in the matching pool. `CatForm.clean()` also prevents a sterilized cat from enabling the mode.



Agregar certificado médico 🐾 Cruce responsable

Estado para cruce responsable:

- Vacuna vigente
- Certificado médico registrado
- No esterilizado

El certificado debe ser emitido por un veterinario. Solo los gatos con vacuna vigente y certificado médico pueden aparecer en el modo cruce responsable.

Figure 15. Editar Perfil screen that contains all three requirements for “Cruce responsable”.

Agregar certificado médico Cruce responsable

Estado para cruce responsable:

- ☒ Sin vacuna vigente
- ☐ Certificado médico registrado
- ☐ No esterilizado

Figure #. Editar Perfil screen that with vaccine unavailable for “Cruce responsable”.

HU-11 – Medical History Registration

As: a cat owner

I need: to register my cat's medical history

So that: the responsible mating mode validation is met

Status: **Implemented**

Module: `cats/views.py` → `editar_perfil` · `cats/forms.py` → `MedicalRecordForm`

Evidence:

MedicalRecordForm is optional – ignored if the diagnosis field is blank. If filled, a document attachment is available. Records are listed with a "View" link to the PDF/image and a delete button. A record with an attached document counts as the veterinary certificate for responsible mating.

Registros médicos (1)

Control medico Ver ✕
30/07/2026

Agregar certificado médico Cruce responsable

Estado para cruce responsable:

- ☒ Sin vacuna vigente
- ☐ Certificado médico registrado
- ☐ No esterilizado

El certificado debe ser emitido por un veterinario. Solo los gatos con vacuna vigente y certificado médico pueden aparecer en el modo cruce responsable.

Diagnóstico o procedimiento

Ej: Revisión general, Desparasitación...

Notas adicionales

Observaciones adicionales del veterinario (opcional)...

Adjuntar certificado (PDF o imagen)

Seleccionar archivo Sin archivos seleccionados

Figure 16. *Editar Perfil* screen showing medical history registration.

HU-07 / HU-08 – Show Profiles and Swipe

As: a cat owner

I need: to view profiles of other cats and swipe right (like) or left (dislike)

So that: I can find compatible candidates efficiently

Status:  **Implemented**

Module: `matching/views.py` → `swipe`, `swipe_like`, `swipe_rechazar`

Evidence:

The swipe view filters candidates: `modo_cruce_responsable=True`, not owned by the authenticated user, not already liked or disliked. The card supports drag-based swiping (Pointer Events API): dragging > 100 px right submits a like, dragging left submits a dislike. The card animates off-screen.

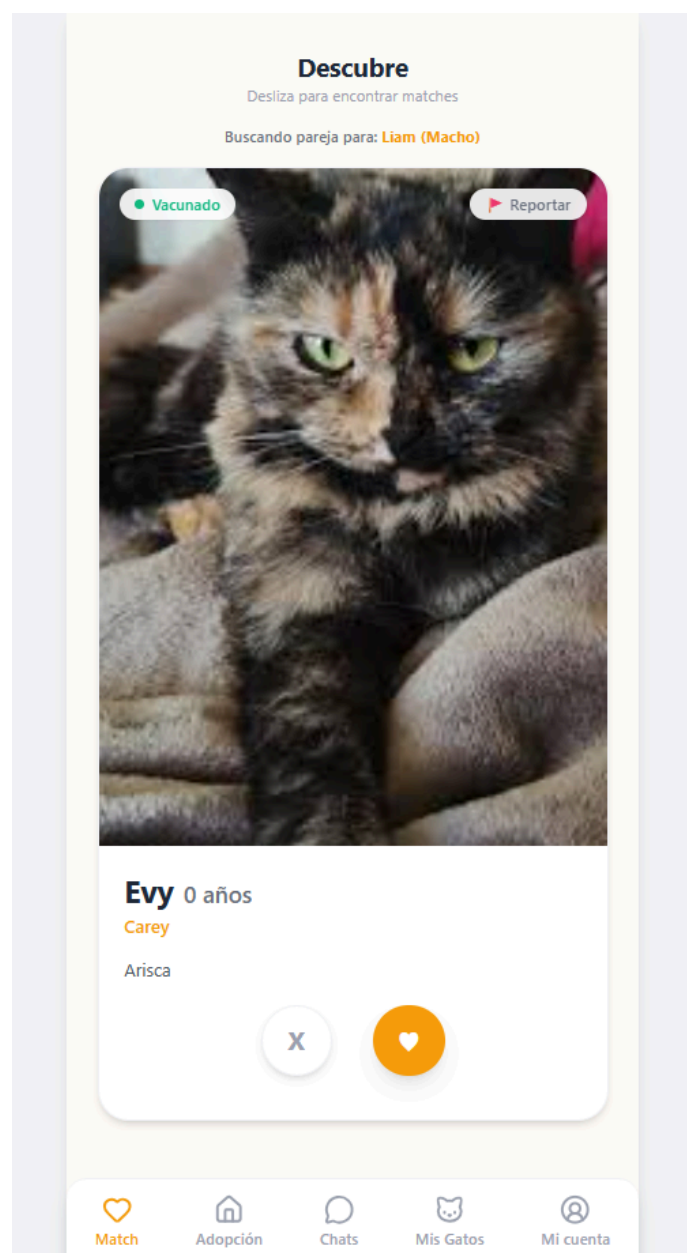


Figure 17. *Match* screen for Liam with another cat.

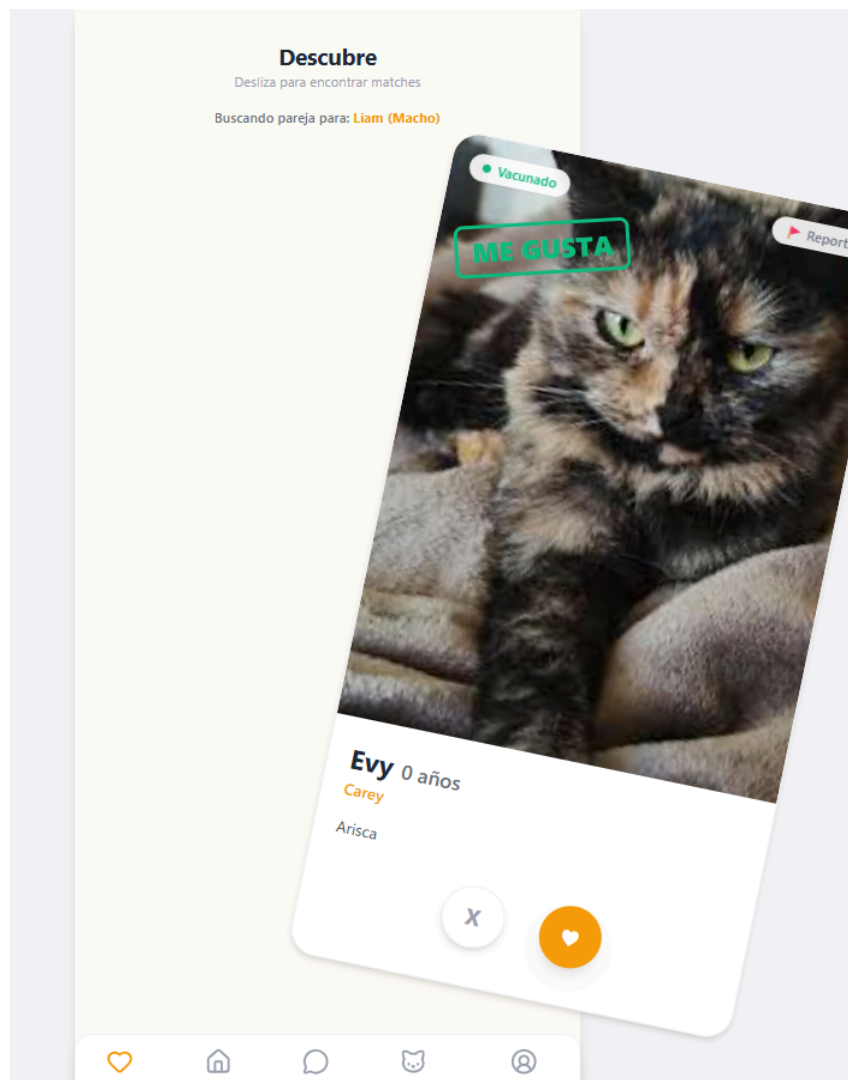


Figure 18. Match screen for Liam with another cat giving it a like.

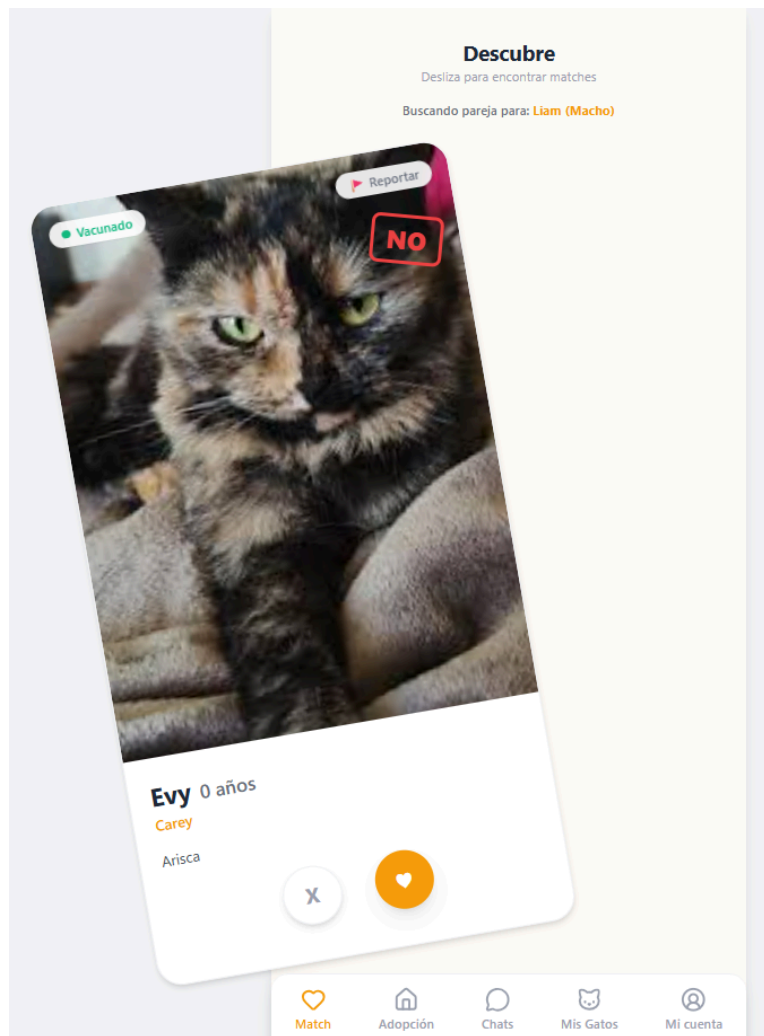


Figure 19. Match screen for Liam with another cat giving it a dislike.

HU-09 – Match Generation

As: a cat owner

I need: to receive a notification when a match is generated with another cat

So that: I know there is mutual interest and can start a conversation

Status:  **Implemented**

Module: `matching/views.py` → `swipe_like` · `chat/signals.py` → `crear_chat_al_aceptar`

Evidence:

`swipe_like` creates a Like, then checks for the reverse Like. If mutual, the Match transitions to `estado="ACEPTADO"`. The Observer signal (`chat/signals.py`) detects this and auto-creates the Chat via `Chat.objects.get_or_create(match=instance)`. The match appears in the user's match list.

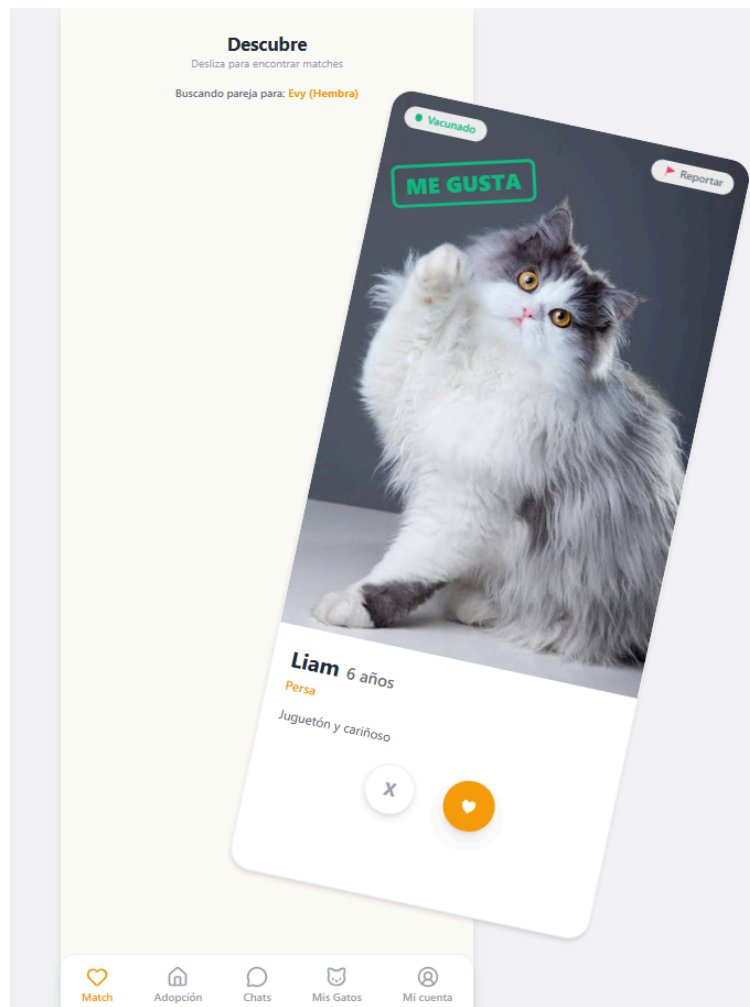


Figure 20. Match screen for Evy with Liam giving it a like.

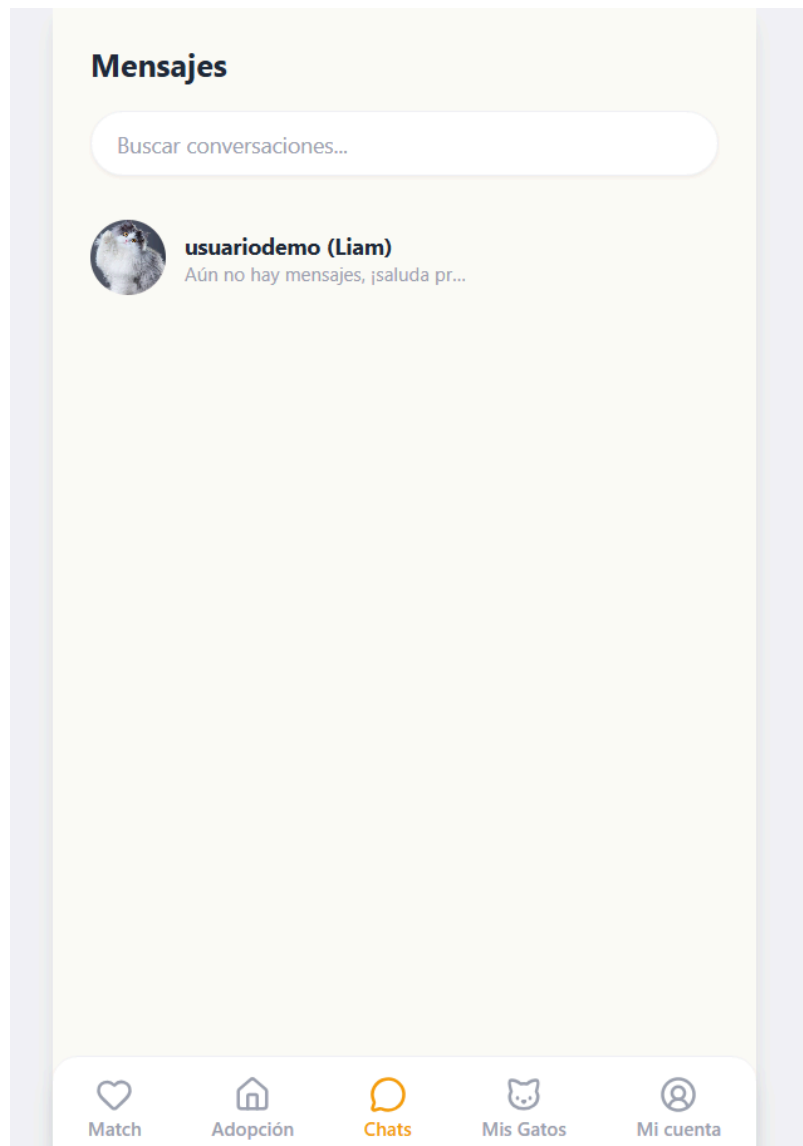


Figure 21. Chats screen for Evy with Liam after giving it a like.

HU-12 – Chat Between Matched Users

As: a cat owner

I need: to send and receive messages and images with owners I matched with

So that: I can coordinate the mating or adoption

Status:  **Implemented**

Module: [chat/views.py](#) → [lista_chats](#), [detalle_chat](#) · [chat/signals.py](#)

Evidence:

The Chat is auto-created by the Observer signal when a Match reaches ACEPTADO. Users can send text messages and image attachments (Message objects). The JS client polls `/chat/mensajes/<chatId>/nuevos/` every 3 seconds and appends new bubbles without reloading the page. A second chat entry point exists from adoption posts.

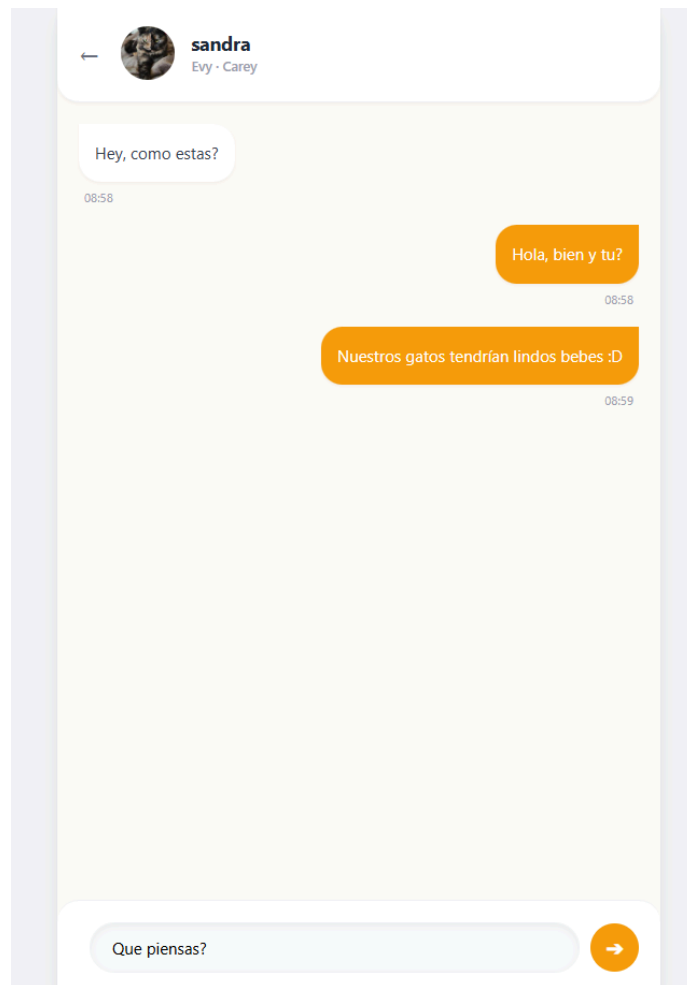


Figure 22. Chat screen of Liam's owner with Evy's.

HU-17 – Account Deletion

As: a cat owner

I need: to delete my account

So that: I have full control over my personal information

Status:  **Implemented**

Module: [users/views.py](#) → [eliminar_cuenta](#)

Evidence:

The `eliminar_cuenta` view requires password confirmation. Sets `is_active=False` instead of hard-deleting, preserving historical data (matches, messages, adoption posts). The deletion form is revealed only after clicking "Delete account" in Mi Cuenta, preventing accidental submissions.

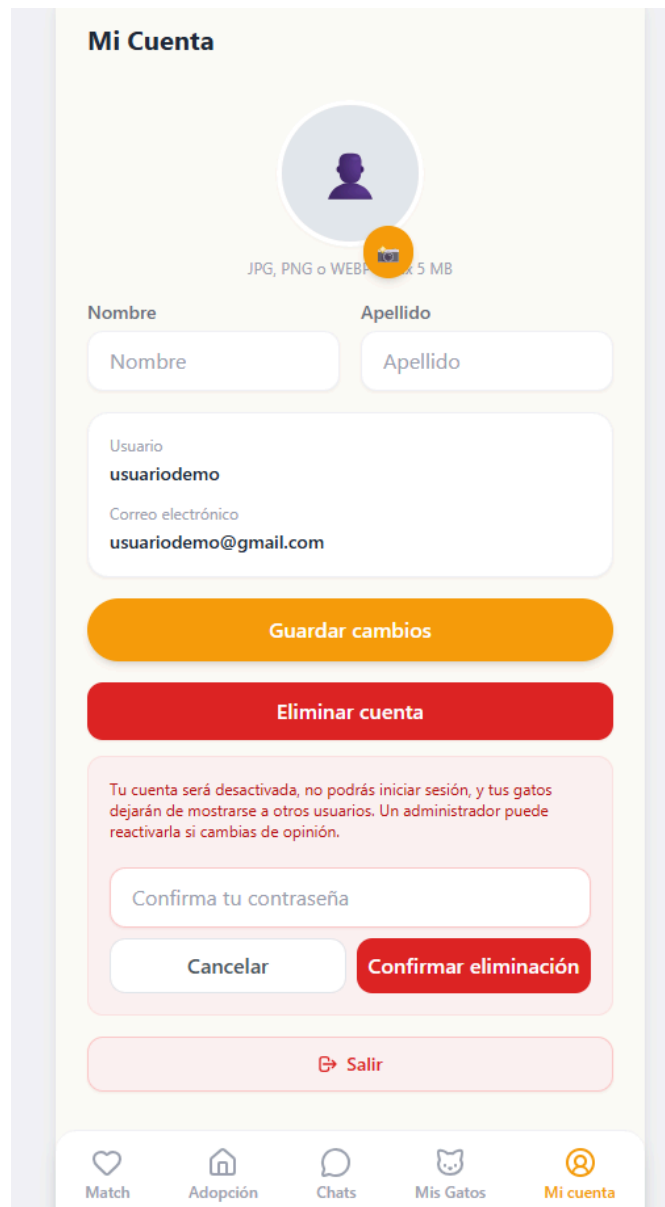


Figure 23. Mi cuenta screen with “Eliminar cuenta” option activated.

HU-29 – Adoption Post Publication

As: a cat owner

I need: to publish a cat in the adoption module

So that: other interested users can request it

Status: **Implemented**

Module: `adoption/views.py` → `lista_publicaciones`, `crear_publicacion`, `editar_publicacion`

Evidence:

The form filters the cat dropdown to show only the user's own cats. Posts have status AVAILABLE / RESERVED / ADOPTED. The public listing excludes the user's own posts and only shows AVAILABLE. `IsGatoOwnerOrStaff` (Strategy) restricts modifications to owner or staff. A "Contact" button opens a chat directly from the post.

← **Publicar en Adopción**

Gato *

Evy

Descripción *

Cría de un cruce responsable

Publicar

The image shows a mobile application screen for posting an adoption. At the top, there is a back arrow and the title 'Publicar en Adopción'. Below this, there are two required fields: 'Gato' and 'Descripción'. The 'Gato' field is a dropdown menu with 'Evy' selected. The 'Descripción' field is a text area containing 'Cría de un cruce responsable'. At the bottom of the form is an orange 'Publicar' button.

Figure 24. Publicar en adopción screen form.

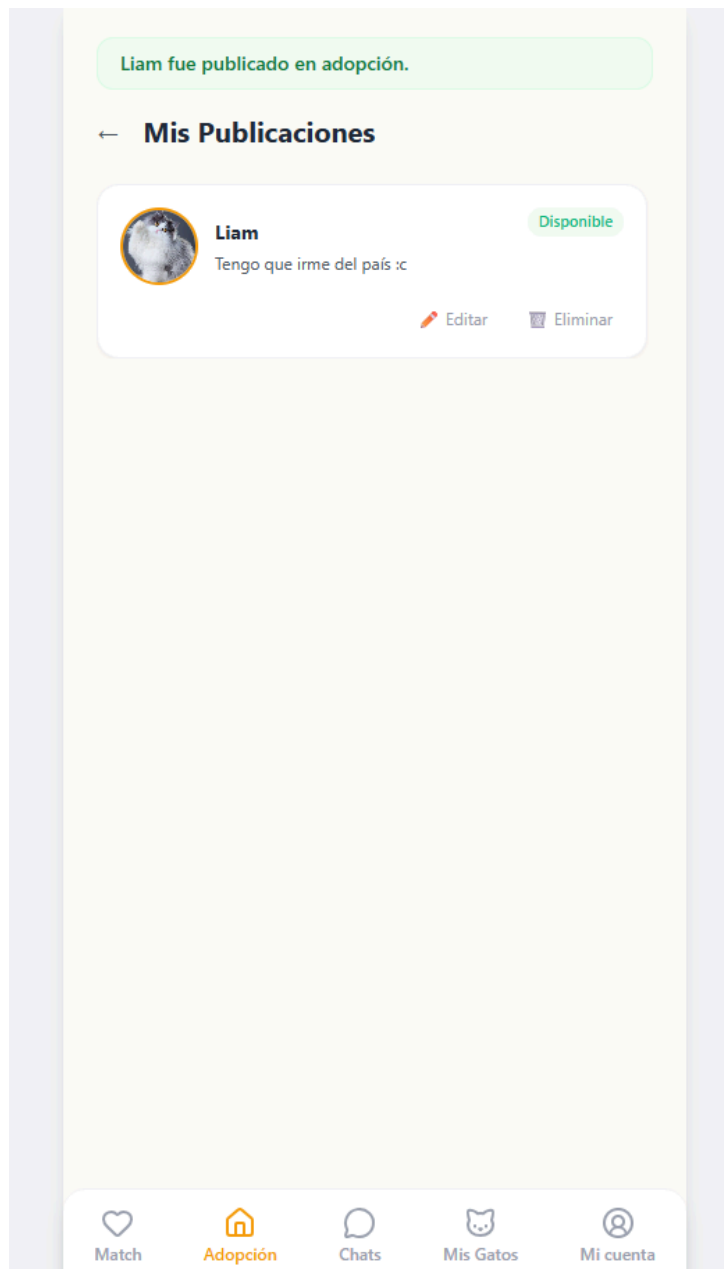


Figure 25. Adopción screen showing my post publication.

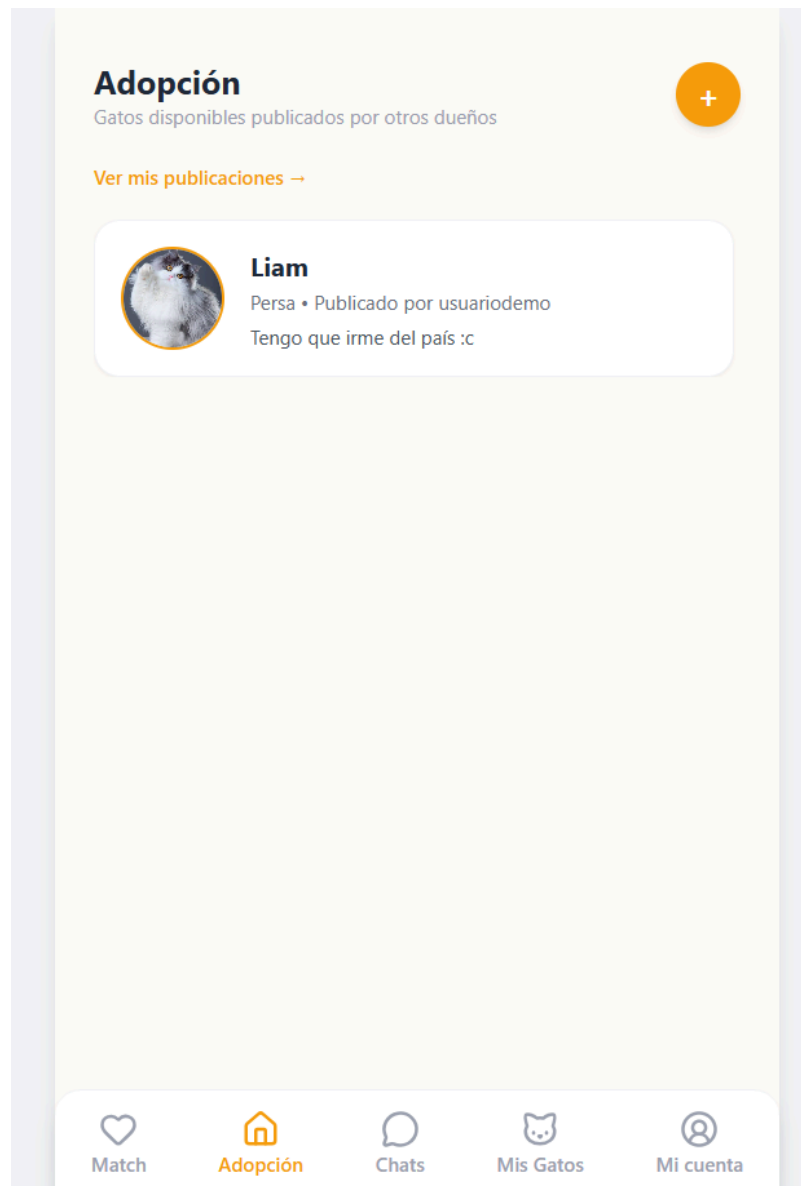


Figure 26. Adopción screen showing another's post publication.

HU-32 – Behavior Reports

As: a cat owner

I need: to report inappropriate behavior by another user

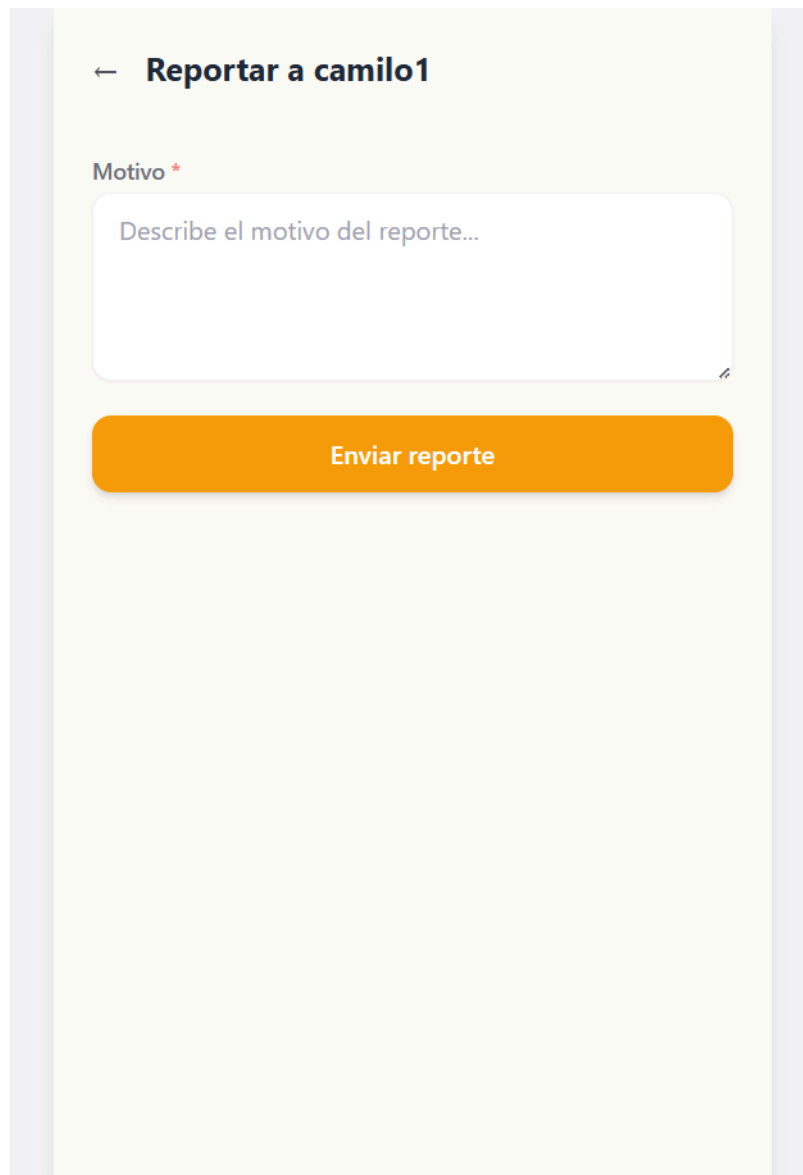
So that: the platform team can review and take action

Status:  **Implemented**

Module: `reports/views.py` → `crear_reporte`, `mis_reportes`

Evidence:

Reports are initiated with the target user pre-filled via URL parameter (`?usuario=<id>`), supplied automatically from cat profiles or chat – there is no free-form user selector. `get_permissions()` (Strategy) ensures only staff can update estado. Regular users see only their own reports; staff sees all.



The image shows a mobile application screen for reporting a user. At the top, there is a back arrow and the title "Reportar a camilo1". Below this, the label "Motivo *" is followed by a text input field with the placeholder text "Describe el motivo del reporte...". At the bottom of the form is an orange button labeled "Enviar reporte".

← **Reportar a camilo1**

Motivo *

Describe el motivo del reporte...

Enviar reporte

Figure 27. Reporting a user form.

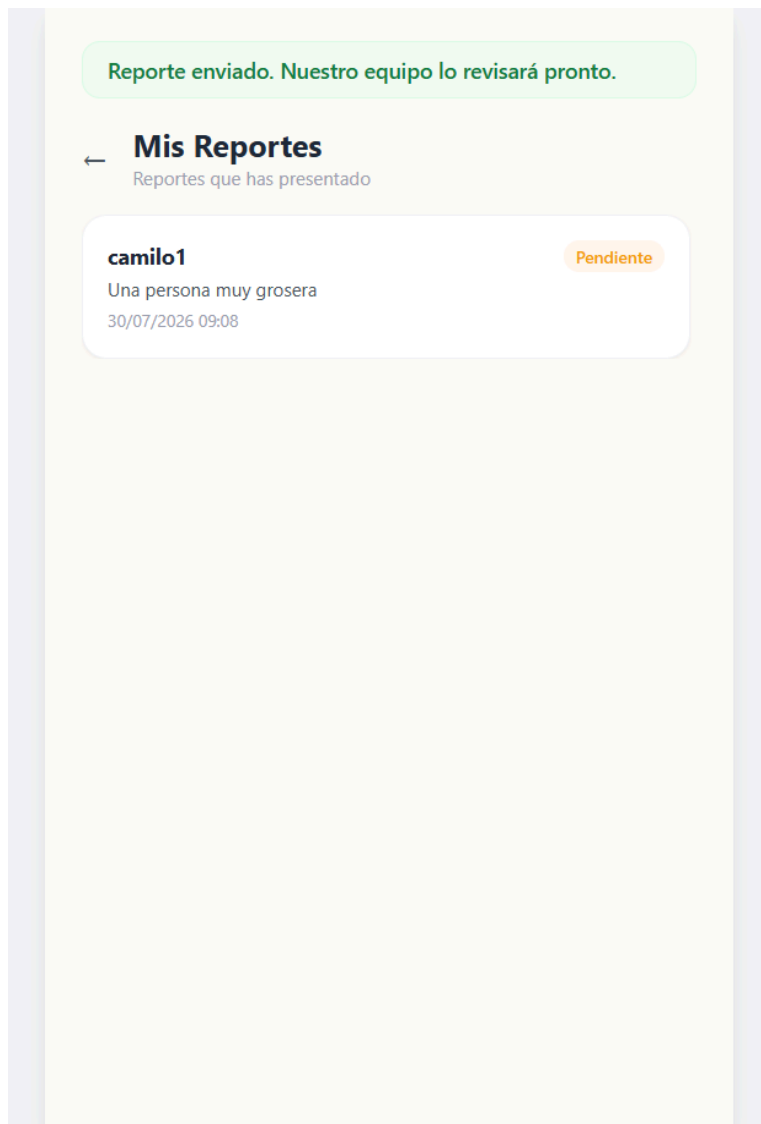


Figure 28. Mis Reportes screen showing historical and successful report.

HU-10 – Password Recovery

As: a cat owner

I need: to recover my password if I have forgotten it

So that: I can regain access to the platform

Status:  **Implemented**

Module: `users/urls.py` → `PasswordResetView`, `PasswordResetConfirmView` (Django built-in)

Evidence:

Uses Django's built-in password reset flow. The user enters their email; Django generates a signed token and, in the current development configuration, prints the reset link to the console (`EMAIL_BACKEND = console`). In production this would be replaced with an SMTP backend.



Recuperar contraseña

Te enviaremos un enlace para restablecer tu contraseña.

Correo electrónico

Enviar enlace de recuperación

[Volver a iniciar sesión](#)

Figure 29. Recuperar contraseña screen form with email information.

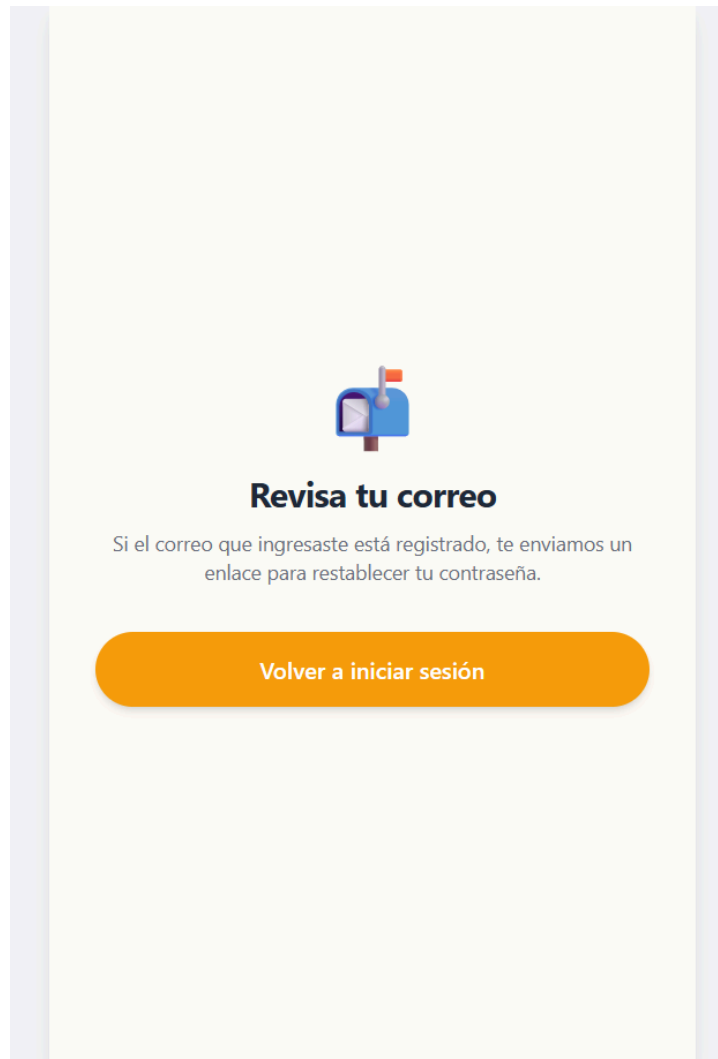


Figure 30. Recuperar contraseña screen with successful email send.

6. API Endpoints

Mewtual exposes both template-based HTML views (server-side rendered) and REST API endpoints via Django REST Framework. The API uses the same session cookie as the web login – no separate JWT tokens. All API endpoints require authentication.

6.1 REST API Resources

Resource	ViewSet	URL Base	Verbs	Permissions
Gatos	CatViewSet (ModelViewSet)	cats/api/	GET POST PUT PATCH DELETE	IsAuthenticated + owner of cat
Usuarios	UserViewSet (ReadOnly)	users/users/	GET	IsAuthenticated
Perfiles	ProfileViewSet (ModelViewSet)	users/profiles/	GET POST PUT PATCH DELETE	IsAuthenticated + owner or staff
Bloqueos	BlockViewSet (ModelViewSet)	users/blocks/	GET POST PUT PATCH DELETE	IsAuthenticated

Resource	ViewSet	URL Base	Verbs	Permissions
Matches	MatchViewSet (ModelViewSet)	matching/matches/	GET POST PUT PATCH DELETE	IsAuthenticated + owner or staff
Chats	ChatViewSet (ModelViewSet)	chat/chats/	GET POST PUT PATCH DELETE	IsAuthenticated
Mensajes	MessageViewSet (ModelViewSet)	chat/messages/	GET POST PUT PATCH DELETE	IsAuthenticated
Publicaciones	AdoptionPostViewSet (ModelViewSet)	api/adoption/adoptions/	GET POST PUT PATCH DELETE	IsAuthenticated + IsGatoOwnerOrStaff
Reportes	ReportViewSet (ModelViewSet)	api/reports/reports/	GET POST PUT PATCH DELETE*	Dynamic per role (staff: update/destroy)

6.2 Template-Based Views (HTML)

Method	URL	Description	Auth
GET/POST	/users/login/	Login form and submission.	No
GET/POST	/users/register/	Registration form and submission.	No
GET	/users/logout/	Ends the current session.	Yes
GET/POST	/users/mi-cuenta/	Edit profile (name, photo) + delete account.	Yes
GET/POST	/users/password-reset/	Request password reset.	No
GET	/matching/home/	Home screen.	Yes
GET	/matching/swipe/	Show next candidate card.	Yes
POST	/matching/swipe/like/<id>/	Register a like.	Yes
POST	/matching/swipe/rechazar/<id>/	Register a dislike.	Yes
GET	/cats/mis_gatos/	List authenticated user's cats.	Yes
GET/POST	/cats/nuevo/	Create cat profile.	Yes
GET/POST	/cats/<id>/editar/	Edit cat profile.	Yes
GET/POST	/cats/<id>/eliminar/	Delete cat profile (confirmation).	Yes
GET	/chat/	List of all chats.	Yes
GET/POST	/chat/<id>/	Chat detail and message submission.	Yes
GET	/adoption/	Public adoption listings.	Yes
GET/POST	/adoption/nueva/	Create an adoption post.	Yes
GET/POST	/reports/nuevo/	Submit a behavior report.	Yes

7. Testing

7.1 Test Framework

The project uses Django's built-in test runner (`python manage.py test`), which runs all `TestCase` classes discovered in each app's `tests.py` file. Tests are executed automatically on every push to main via the CI/CD pipeline (see Section 8).

7.2 Test Categories

Category	Scope	Tools
Unit tests	Individual model methods, view behavior, form validations, permission logic.	Django <code>TestCase</code> , DRF <code>APITestCase</code>
Integration tests	End-to-end flows: HTTP request → view → ORM → DB → response, redirect chains, signal side-effects.	Django test client, DRF <code>APIClient</code>
Linting	Code style and syntax compliance with PEP 8.	Flake8

7.3 Key Test Cases

Test	Module	What it verifies
<code>test_like_with_incoming_pending_match_becomes_mutual</code>	<code>matching/tests.py</code>	Mutual like transitions Match to ACEPTADO and the Observer signal auto-creates the Chat.
<code>test_like_with_no_incoming_match_creates_pending_match</code>	<code>matching/tests.py</code>	A single like creates a PENDIENTE Match; no Chat is created yet.
<code>test_rechazar_with_incoming_pending_match_marks_it_rechazado</code>	<code>matching/tests.py</code>	A dislike sets the Match to RECHAZADO; no Chat is created.
Strategy: owner-only write on adoption	<code>adoption/tests.py</code>	Non-owner cannot PUT/PATCH/DELETE an <code>AdoptionPost</code> ; owner and staff can.
Strategy: staff-only estado update on reports	<code>reports/tests.py</code>	Regular user cannot update estado; staff user can.

7.4 Linter Testing with Flake8

1. Activate the virtual environment and navigate to the backend/ folder.

2. Install Flake8:

```
pip install flake8
```

3. Create a `.flake8` configuration file in the project root:

```
[flake8]
max-line-length = 120
exclude = migrations/, __pycache__/, venv/, .env
```

4. Run Flake8 and generate a report:

```
flake8 --output-file=flake8_report.txt
```


Results: the initial run found 52 errors, primarily unused imports (F401), missing blank lines between functions (E302/E303), trailing whitespace (W291/W293), and imports not at the top of the file (E402). After corrections the count dropped to 9. The remaining 9 errors correspond to `django.test.TestCase` imports in the auto-generated tests.py files that had not yet been used – they are not production code issues.

Errors found in `settings.py` were intentionally ignored – this file is auto-generated by Django and is not part of the team's code.

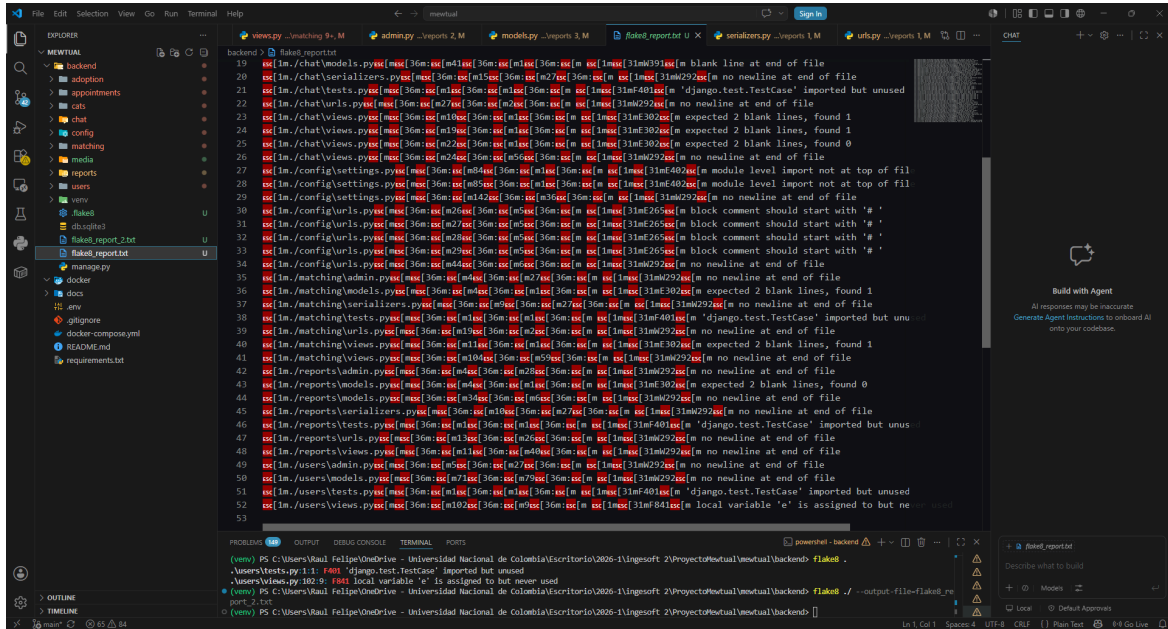


Figure 31. Archive of error reports found in the backend.

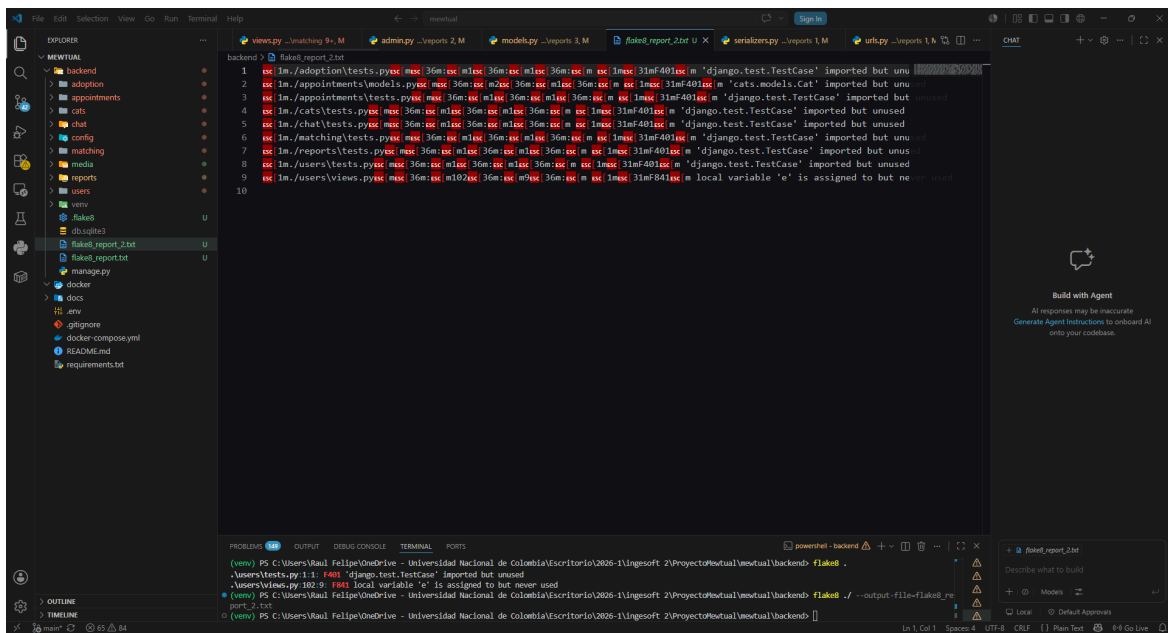


Figure 32. Archive of error reports found in the backend after corrections.

8. CI/CD Pipeline

8.1 Overview

The project includes a GitHub Actions workflow that automatically runs migrations and the full test suite on every push and pull request to the main branch. This ensures regressions are caught before code is merged.

8.2 Workflow Configuration

File location: `.github/workflows/ci.yml`

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      DB_NAME: mewtual
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_HOST: localhost
      DB_PORT: 5432

    services:
      db:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: mewtual
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"
          cache-dependency-path: requirements.txt
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run migrations
        working-directory: Final-Project/backend
```

```
run: python manage.py migrate
- name: Run tests
  working-directory: Final-Project/backend
  run: python manage.py test
```

8.3 Pipeline Stages

Stage	Description
Checkout	Checks out the repository code at the triggering commit.
Set up Python 3.12	Configures Python 3.12 with pip cache using requirements.txt as cache key.
Install dependencies	Installs all packages from requirements.txt (Django, DRF, Pillow, psycopg2-binary, etc.).
Run migrations	Applies all database migrations against the PostgreSQL service container.
Run tests	Executes the full Django test suite (python manage.py test). Fails the build if any test fails.

8.4 Infrastructure

The workflow spins up a PostgreSQL 16-alpine container as a service with health checks (pg_isready every 10 seconds, 5 retries). Database credentials are injected as environment variables — the same variables read by settings.py via python-dotenv. This mirrors the local development setup exactly, so tests run in an identical environment.

9. Setup and Execution Instructions

9.1 Prerequisites

- Python 3.10 or higher
- Docker and Docker Compose
- Git

9.2 Instructions

1. Clone the Repository

```
git clone https://github.com/rrodriguezhe/mewtual.git
cd mewtual
```

2. Configure Environment Variables

Create a `.env` file inside the Final-Project folder (same level as the backend folder):

```
DB_NAME=mewtual_db
DB_USER=mewtual_user
DB_PASSWORD=your_secure_password
DB_HOST=localhost
DB_PORT=5432
```

 The `.env` file must be at `mewtual/Final-Project/.env` — `settings.py` loads it from `BASE_DIR.parent / ".env"`.

3. Open Docker Desktop and Start the Database

```
docker compose up -d
```

4. Create and Activate the Virtual Environment

```
cd Final-Project/backend
python -m venv venv
# Windows PowerShell:
.\venv\Scripts\Activate.ps1
# macOS / Linux:
source venv/bin/activate
```

5. Install Dependencies

```
pip install -r requirements.txt
```

6. Apply Database Migrations

```
python manage.py migrate
```

7. (Optional) Create a Superuser

```
python manage.py createsuperuser
```

8. Run the Development Server

```
python manage.py runserver
```

The application will be available at <http://127.0.0.1:8000/>

9. Run Tests

```
python manage.py test
```

10. Run Flake8 Linter

```
flake8 --output-file=flake8_report.txt
```



For more details, refer to the [README.md](#) at the repository root.

10. Reflection and Evaluation

10.1 Strengths

- Modular architecture: each Django app is independently developed and tested without breaking other modules. The config app orchestrates all apps without any app needing to know about the others.
- Data integrity: CASCADE relationships (Cat → Vaccine, Cat → MedicalRecord) ensure consistent deletion. Account deactivation (`is_active=False`) preserves historical data while preventing login.
- Design patterns in real code: the Strategy pattern (`IsGatoOwnerOrStaff`, `get_permissions()`) and Observer pattern (`post_save` signal auto-creating Chat) were identified from real architectural constraints, not retrofitted.
- Form validation: ModelForms with custom `clean()` methods centralize validation. Cross-field rules (sterilized cat cannot enable mating mode) are enforced at the form level.
- User data isolation: all queries filter by `owner=request.user`, so users never access another user's data – a 404 is returned instead.
- CI/CD: automated test execution on every push ensures regressions are caught before merging to main.
- Chat without page reload: JavaScript polling every 3 seconds appends new messages without reloading, giving a near-real-time feel without WebSockets.

10.2 Limitations

- No real-time features: chat uses JS polling (`setInterval 3s`). Under high load this generates unnecessary requests. Django Channels + WebSockets would be the production solution.
- No email sending in development: password reset emails are printed to the console. An SMTP backend must be configured for production.
- Appointment scheduling incomplete: the appointments app is wired up but the core scheduling logic and automatic reminders are not yet implemented.
- Favorites not surfaced in UI: the Favorite model exists but no view or template was built to add or display favorite cats.
- No geolocation: the map of nearby cats was planned but not implemented in the current sprint.

10.3 Future Improvements

- Replace JS polling in chat with WebSocket-based real-time messaging using Django Channels and Redis.
- Configure an SMTP backend for production email (password reset, match notifications).
- Complete the appointments module: propose, confirm, and send automatic reminders via Celery + Beat.
- Build the favorites UI (add/remove buttons and a favorites list screen).
- Integrate geolocation using the browser Geolocation API and store coordinates on the Cat model.
- Add a compatibility percentage algorithm based on breed, age, and health criteria.

10.4 Lessons Learned

- Django's `@login_required` decorator requires explicitly setting `LOGIN_URL` in `settings.py` when the project uses a custom auth URL – otherwise the default `/accounts/login/` path causes a 404.
- Optional sub-forms (`VaccineForm`, `MedicalRecordForm`) require setting `required=False` on all fields and checking for data presence before saving.

- The Observer pattern via Django signals keeps the matching → chat dependency strictly one-directional without any extra infrastructure – the signal dispatcher is already built in.
- Git branch discipline is essential when five developers share a modular monolith – clear app ownership prevents merge conflicts.
- The .env file location must match what load_dotenv() expects; moving the folder structure mid-project can break database connections silently.

11. References

- Django Software Foundation. (2024). Django Documentation (v6.x). <https://docs.djangoproject.com/>
- Django REST Framework. (2024). DRF Documentation. <https://www.django-rest-framework.org/>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- Docker Inc. (2024). Docker Documentation. <https://docs.docker.com/>
- GitHub. (2024). GitHub Actions Documentation. <https://docs.github.com/en/actions>
- Flake8 Team. (2024). Flake8 Documentation. <https://flake8.pycqa.org/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Tailwind CSS. (2024). Tailwind CSS Documentation. <https://tailwindcss.com/docs/>
- Pillow Contributors. (2024). Pillow Documentation. <https://pillow.readthedocs.io/>
- Figma Inc. (2024). Figma Documentation. <https://help.figma.com/>